P01

LINUX Y ENTORNOS VIRTUALES DE PYTHON

SISTEMAS OPERATIVOS PARA PRÁCTICAS

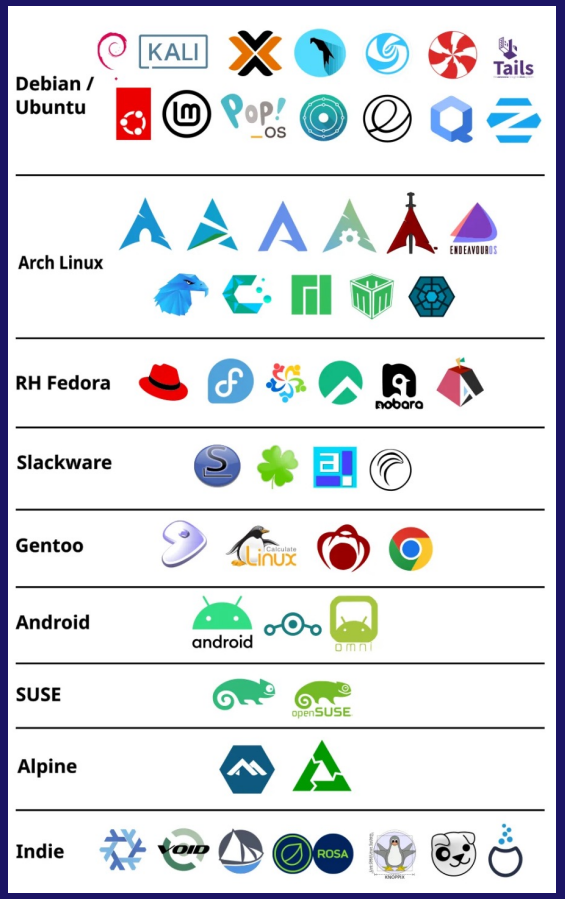
- No usaremos Windows
- Usaremos GNU/Linux (cualquier distro, Ubuntu 24.04 recomendada) o macOS (recomendado macOS 15 Sequoia)
- ¿Qué hago si solo tengo Windows? Opciones de mejor a peor:
  - Opción 1: Instala Linux (puedes tener Windows y Linux a la vez)
    - https://www.softzone.es/windows/como-se-hace/ubuntu-windows-dual-boot/
  - Opción 2: Usa una máquina virtual
    - https://www.dlsi.ua.es/~materuel/vm.html (password: prog2)
  - Opción 3: Instala WSL
    - https://learn.microsoft.com/es-es/windows/wsl/install

GNU/LINUX

- GNU/Linux es una familia de sistemas operativos Open Source
- Originalmente, fue concebido y creado como un pasatiempo por Linus Torvalds en 1991
- Mientras Linus estaba en la universidad, intentó crear una versión Open Source, alternativa y gratuita del sistema operativo MINIX, que a su vez se basaba en los principios y el diseño de Unix
- Ese pasatiempo logró convertirse en el sistema operativo con la mayor base de usuarios, el más usado en los servidores de Internet

DISTRIBUCIÓN GNU/LINUX

Una distribución GNU/Linux (coloquial y abreviadamente llamada distro) es una distribución de software basada en el núcleo Linux, y a menudo, un Sistema de gestión de paquetes que incluye determinados paquetes de software, para satisfacer las necesidades de un grupo específico de usuarios, dando así origen a ediciones domésticas, empresariales y para servidores.



POSIX

- POSIX es una serie de normas, estándares y convenciones creadas por la comunidad de usuarios de Unix.
- Se utilizan con el fin de mejorar la compatibilidad entre diferentes sistemas operativos Unix y proporcionar una base para que las aplicaciones sean más portables entre distintos sistemas operativos.
- POSIX define una interfaz estándar del sistema operativo y el entorno, incluyendo un intérprete de comandos (o "shell"), y programas de utilidades comunes para apoyar la portabilidad de las aplicaciones a nivel de código fuente.
- Linux y macOS son sistemas POSIX. Windows no

COMANDOS DE DIRECTORIOS

Comando	Significado	Ejemplo
pwd	Muestra la ruta actual	pwd
cd [ruta]	Cambia el directorio actual a *ruta*	cd /
ls	Muestra el contenido de la ruta actual (admite parámetros como -l -a -t -S, o comodines como *, ?, [])	ls -la *.py
cp [origen] [destino]	Copia el fichero *origen* a la ruta *destino*	cp main.py /home/usuario/Desktop
mv [origen] [destino]	Mueve (o cambia el nombre) el fichero *origen* a la ruta *destino*	mv main.py backup.py
mkdir [directorio]	Crea el directorio *directorio*	mkdir nuevo_directorio
rmdir [directorio]	Elimina un directorio **vacio**	rmdir nuevo_directorio
rm	Elimina un fichero o directorio. Admite parámetros **peligrosos:** **-r** (recursivo) y **-f** (directorio)	rm main.py

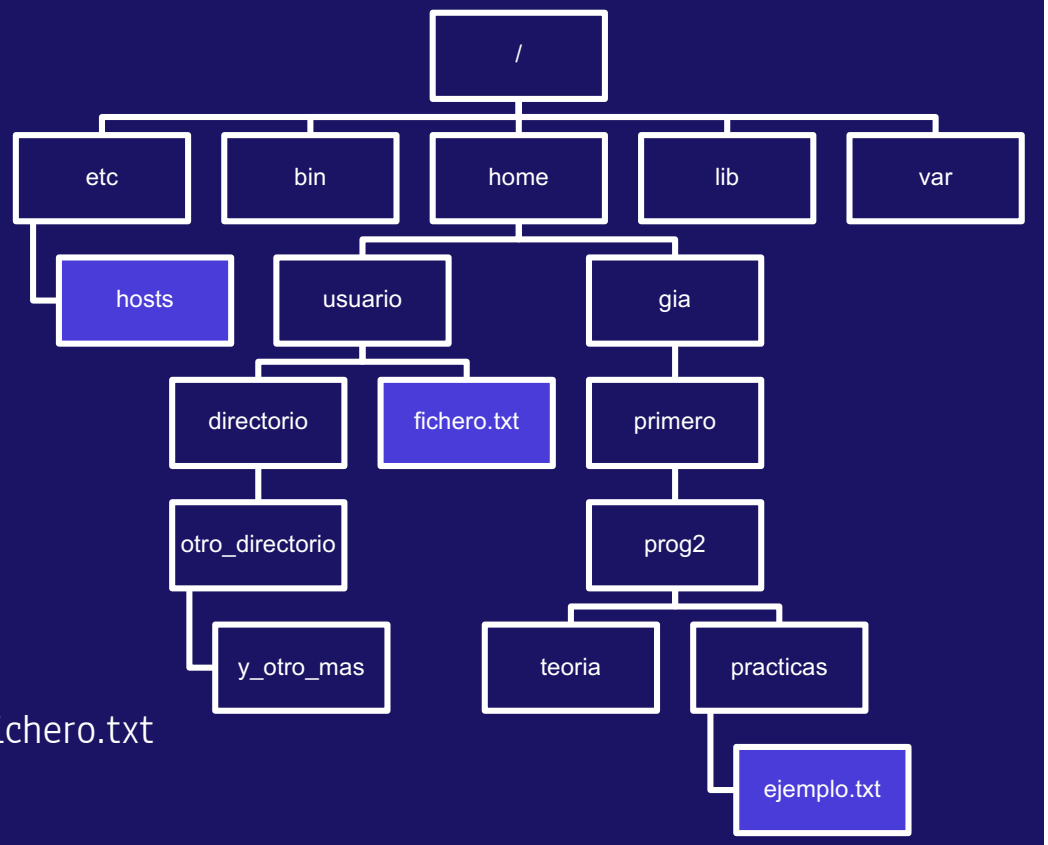
RUTAS

- Raíz del sistema de archivos: /
  - En POSIX no hay "unidades" como en Windows (C:)
  - Si queremos situarnos en la raíz: cd /

- Directorio personal:
  - /home/tu_usuario (Linux)
  - /Users/tu_usuario (macOS)
  - Acceder al directorio personal: cd (sin parámetro)

- Movernos por directorios:
  - Subir un nivel: cd .. (espacio obligatorio)
  - Bajar un nivel: cd [directorio]
  - Navegar a una ruta concreta: cd [ruta]

- Rutas relativas a un fichero /home/usuario/directorio/fichero.txt
  - Desde /home → usuario/directorio/fichero.txt
  - Desde /home/usuario/directorio → fichero.txt
  - Desde /home/usuario/directorio/otro_directorio → ../fichero.txt
  - Desde /home/usuario/directorio/otro_directorio/y_otro_mas → ../../fichero.txt

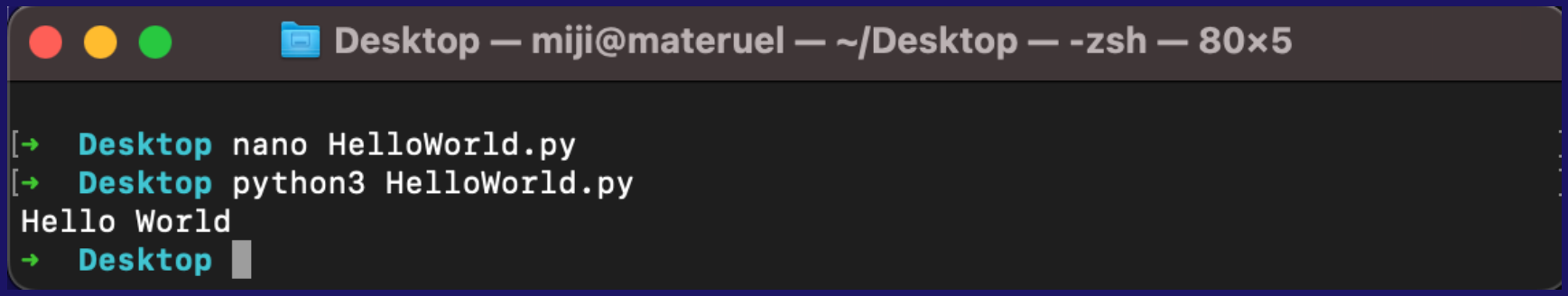


EJERCICIO

1. Abre el terminal
2. Comprueba en qué ruta estás actualmente
3. Muestra el contenido de ese directorio
4. Muestra también los ficheros ocultos (-a). ¿Con qué carácter empieza el nombre de un fichero oculto en POSIX?
5. Crea un directorio llamado *gia* en tu directorio personal
6. Crea un directorio llamado *primero* en *gia*
7. Crea un directorio llamado *prog2* en primero
8. Crea un directorio llamado *teoria* en *prog2*
9. Crea un directorio llamado *practicas* en *prog2*
10. Copia el fichero */etc/hosts* a tu directorio personal
11. Renombra *hosts* como *ejemplo.txt*
12. Mueve *ejemplo.txt* a practicas
13. Haz una copia de *ejemplo.txt* llamada *ejemplo2.txt*
14. Elimina el directorio vacío *teoria*
15. Elimina el directorio *gia* con un solo comando (y con mucho cuidado)

COMANDOS DE FICHEROS PARA PYTHON

Comando	Significado	Ejemplo
nano [fichero]	Crea o edita el fichero *fichero* (salimos con control + X)	nano test.py
cat [fichero]	Muestra por pantalla el contenido del fichero *fichero*	cat test.py
python	Inicia el intérprete de Python en modo interactivo REPL (Read Evaluate Print Loop). En macOS, o si tenemos Python 2 y Python 3 a la vez, el comando puede llamarse *python3*	python
python [fichero]	Ejecuta el fichero de Python *fichero* en modo script	python test.py



EJERCICIO

1. Inicia una sesión REPR de Python
2. Ejecuta un "Hello World!" en modo REPL
3. Sal del intérprete de Python llamando a la función *exit()*
4. Escribe un "Hello World!" dentro de un fichero llamado *hello_world.py* usando *nano*
5. Comprueba el contenido del fichero con *cat*
6. Ejecuta *hello_world.py* en modo script

¿QUÉ ES UN VENV?

- El módulo *venv* de Python permite crear "entornos virtuales" ligeros, cada uno con su propio conjunto independiente de paquetes Python.

- Un entorno virtual se crea sobre una instalación de Python existente, conocida como Python "base" del entorno virtual, y puede opcionalmente aislarse de los paquetes del entorno base, de modo que sólo estén disponibles aquellos instalados explícitamente en el entorno virtual.

- Cuando se utilizan desde dentro de un entorno virtual, las herramientas de instalación comunes como pip instalarán paquetes Python en un entorno virtual sin necesidad de que se les diga explícitamente que lo hagan.

¿QUÉ ES UN VENV?

- Se utiliza para contener un intérprete específico de Python y bibliotecas y binarios de software que son necesarios para dar soporte a un proyecto (biblioteca o aplicación). Por defecto, están aislados del software de otros entornos virtuales y de los intérpretes y bibliotecas de Python instalados en el sistema operativo.

- Están contenidos en un directorio, convencionalmente llamado venv o .venv en el directorio del proyecto, o bajo un directorio contenedor para muchos entornos virtuales, como ~/.virtualenvs.

- No se sube en sistemas de control de código fuente como Git (podemos usar .gitignore).

- Considerado como desechable - debería ser sencillo borrarlo y recrearlo desde cero. No se coloca ningún código del proyecto en el entorno.

- No se considera como movible o copiable - simplemente se recrea el mismo entorno en la ubicación de destino.

CREACIÓN DE UN VENV

- La creación de entornos virtuales se realiza ejecutando el comando *venv*:

```
python -m venv [ruta al entorno virtual]
```

- Una vez creado el entorno virtual, podemos activarlo. Ejecutamos

```
source [ruta al entorno virtual]/bin/activate
```

- Podemos asegurarnos que estamos usando el ejecutable de Python correcto con *which*:

```
which python
```

- Para desactivar un entorno virtual, escribimos:

```
deactivate
```

CREACIÓN DE UN VENV

Desktop — miji@MBP-M1-Max — ~/Desktop — -zsh — 80×24

```
→  Desktop which python3
/Library/Frameworks/Python.framework/Versions/3.12/bin/python3
→  Desktop python3 -m venv mi_entorno_mirtual
→  Desktop source mi_entorno_mirtual/bin/activate
(mi_entorno_mirtual) →  Desktop which python3
/Users/miji/Desktop/mi_entorno_mirtual/bin/python3
(mi_entorno_mirtual) →  Desktop pip list
Package Version
------- -------
pip     24.0

[notice] A new release of pip is available: 24.0 -> 24.2
[notice] To update, run: pip install --upgrade pip
(mi_entorno_mirtual) →  Desktop deactivate
→  Desktop which python3
/Library/Frameworks/Python.framework/Versions/3.12/bin/python3
→  Desktop
```

activo

no activo

GESTIÓN DE PAQUETES CON PIP

- Se pueden instalar, actualizar y eliminar paquetes usando un programa llamado pip. Podemos navegar por el Índice de Paquetes Python desde tu navegador web en https://pypi.org

- pip tiene varios subcomandos: "install", "uninstall", "freeze", etc.

- Podemos instalar la última versión de un paquete especificando su nombre:

VENV

```
(tutorial-env) $ python -m pip install novas
Collecting novas
  Downloading novas-3.1.1.3.tar.gz (136kB)
Installing collected packages: novas
  Running setup.py install for novas
Successfully installed novas-3.1.1.3
```

- También se puede instalar una versión concreta de un paquete indicando el nombre del paquete seguido de == y el número de versión:

```
(tutorial-env) $ python -m pip install requests==2.6.0
Collecting requests==2.6.0
  Using cached requests-2.6.0-py2.py3-none-any.whl
Installing collected packages: requests
Successfully installed requests-2.6.0
```

GESTIÓN DE PAQUETES CON PIP

- Si se vuelve a ejecutar este comando, pip se dará cuenta de que la versión solicitada ya está instalada y no hará nada. Es posible proporcionar un número de versión diferente para obtener esa versión, o se puede ejecutar **`python -m pip install --upgrade`** para actualizar el paquete a la última versión:

```
(tutorial-env) $ python -m pip install --upgrade requests
Collecting requests
Installing collected packages: requests
  Found existing installation: requests 2.6.0
    Uninstalling requests-2.6.0:
      Successfully uninstalled requests-2.6.0
Successfully installed requests-2.7.0
```

- **`python -m pip uninstall`** seguido de uno o más nombres de paquetes eliminará los paquetes del entorno virtual.

GESTIÓN DE PAQUETES CON PIP

- **`python -m pip show`** mostrará información sobre un paquete en particular:

```
(tutorial-env) $ python -m pip show requests
---
Metadata-Version: 2.0
Name: requests
Version: 2.7.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.com
License: Apache 2.0
Location: /Users/akuchling/envs/tutorial-env/lib/python3.4/site-packages
Requires:
```

- **`python -m pip list`** mostrará todos los paquetes instalados en el entorno virtual:

```
(tutorial-env) $ python -m pip list
novas (3.1.1.3)
numpy (1.9.2)
pip (7.0.3)
requests (2.7.0)
setuptools (16.0)
```

GESTIÓN DE PAQUETES CON PIP

- **`python -m pip freeze`** producirá una lista similar de los paquetes instalados, pero la salida utiliza el formato que python -m pip install espera. Una convención común es poner esta lista en un archivo requirements.txt:

```
(tutorial-env) $ python -m pip freeze > requirements.txt
(tutorial-env) $ cat requirements.txt
novas==3.1.1.3
numpy==1.9.2
requests==2.7.0
```

- A continuación, el archivo **`requirements.txt`** puede añadirse a GIT como parte de una aplicación. Los usuarios pueden entonces instalar todos los paquetes necesarios con **`install -r`**:

```
(tutorial-env) $ python -m pip install -r requirements.txt
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
  ...
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
  ...
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
  ...
Installing collected packages: novas, numpy, requests
  Running setup.py install for novas
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0
```

EJERCICIO

1. Crea un entorno virtual en tu directorio personal
2. Actívalo
3. Comprueba la ruta del ejecutable de Python
4. Comprueba los paquetes que tienes instalados en el entorno virtual
5. Instala el paquete *bson* en tu entorno virtual
6. Comprueba que se ha instalado
7. Comprueba que NO se ha instalado en el intérprete de Python nativo (no virtual)
8. Ejecuta el *hello_world.py* en tu entorno virtual
9. Genera un fichero *requirements.txt* de tu entorno virtual
10. Desinstala el paquete *bson* de tu entorno virtual
11. Restaura los paquetes usando el fichero requirements.txt
12. Desactiva tu entorno virtual